PRODUCT CASE STUDY

ReliefSync: automating disaster relief coordination

A Salesforce-based system that matches relief requests with volunteers and supplies, with no manual dispatching.

B.Tech minor project, JECRC University · Team of 2 · My role: testing & QA

Salesforce Flow Builder · 2025–26

TL;DR

Relief teams often coordinate through calls, sheets and paperwork, so requests sit unattended and volunteers get double-booked. ReliefSync centralizes requests, volunteers and supplies in one system, and a record-triggered flow decides what happens the moment a request is created. In testing, all 12 test cases passed, including shortage and duplicate-assignment scenarios.

4
custom objects: Request, Volunteer, Resource, Assignment

3
automatic outcomes: Assigned, Waiting for Resource, Waiting for Volunteer

12/12
test cases passed, including edge cases

1. The problem

In an emergency, someone has to answer three questions fast: *Do we have the supplies? Is anyone free? Who is going?* Done manually, this causes:

- Delayed response, because coordinators check stock and availability by hand.
- Double-booked volunteers and over-promised supplies.
- No single view of what is pending, assigned or stuck.

Goal: shorten the time from “request raised” to “help assigned” and remove the common failure modes without adding coordinator workload.

2. Users

USER	NEEDS	PAIN TODAY
Relief Coordinator	Log a request quickly and know its status	Chasing people to confirm who is available
Administrator	Oversight of volunteers, stock and all requests	Scattered records, no real-time picture
Volunteer	Clear assignments, no overlapping tasks	Being assigned several tasks at once

3. The solution

Four linked objects hold the data. When a Relief Request is created, a record-triggered flow checks stock, checks volunteers and acts on the result.

1. A new relief request is created and the flow starts automatically.

2. Is resource quantity above 0? If not, status becomes **Waiting for Resource**.
3. Is a volunteer available? If not, status becomes **Waiting for Volunteer**.
4. An Assignment record is created.
5. The volunteer is set to Busy.
6. Resource quantity is reduced by 1.
7. Request status becomes **Assigned**.

Data model: Assignment is a junction object linking one Request, one Volunteer and one Resource (one-to-many from each parent).

---

4. Requirements: user stories

AS A...	I WANT...	ACCEPTANCE CRITERIA
Coordinator	to log a relief request with type, severity and location	Request saves and the assignment flow starts automatically
Coordinator	the system to assign help automatically	If stock and a volunteer exist, an Assignment is created and status becomes Assigned
Coordinator	to see why a request is stuck	Status shows Waiting for Resource or Waiting for Volunteer
Volunteer	not to be double-booked	Assigned volunteer becomes Busy and is skipped by later requests
Administrator	accurate inventory	Resource quantity decreases on every assignment
Administrator	a record of every assignment	Each Assignment links its request, volunteer and resource

5. Prioritization (MoSCoW)

Must have (built)

Request intake, availability checks, auto-assignment, volunteer Busy status, stock reduction, status updates

Should have (next)

Severity-aware ordering, email/SMS notifications, dashboards

Could have

Skill matching, location matching, mobile access

Won't have (for now)

AI prioritization, government system integration, payments

6. Key decisions and trade-offs

- **Declarative automation over code.** Flow Builder let two students ship quickly and keeps the logic readable by non-developers. Trade-off: less flexibility at scale.
- **Availability-only matching.** The v1 flow considers whether a volunteer is free, not their skills or distance. That kept the logic testable but means a High-severity request is not yet handled ahead of a Low one.
- **Check stock before volunteers.** It gives a clear status when either is missing, which is what a coordinator needs to act on.

7. Testing and results (my contribution)

I owned the testing process: I designed and ran 12 manual test cases covering the happy path (assignment, status, stock reduction), shortages (no resource, no volunteer), duplicate prevention and data consistency. All passed after debugging flow configuration issues. Observed flow execution was a few seconds end to end on a Developer Edition org; this was small-scale testing, not a load test.

---

8. How I would measure success

Proposed metrics for a real deployment. The project was not run in the field.

METRIC	WHY IT MATTERS
Time from request to assignment	Core speed outcome
% of requests auto-assigned vs waiting	Automation effectiveness and supply gaps
Volunteer utilization and double-bookings (target 0)	Adoption and correctness
Stock-outs per week	Inventory health
Weekly active coordinators	Engagement and retention

---

9. What I would build next (RICE)

Illustrative scoring. Reach 1–10, Impact 1–3, Confidence %, Effort in person-weeks.

FEATURE	R	I	C	E	SCORE
Severity-based request queue	9	3	80%	2	**10.8**
Email/SMS notifications	9	2	80%	2	**7.2**

Dashboards and reports	6	2	90%	2	**5.4**
Skill-based matching	7	2	80%	3	**3.7**
Location-based matching	8	3	50%	5	**2.4**
Mobile app	8	2	60%	8	**1.2**

Severity ordering comes first because it fixes the largest gap in v1 at low effort. Location matching has higher impact but lower confidence and cost, so it follows once notifications and data are in place.

---

10. What I learned

- Turning a messy operational process into explicit rules (BRD/FRD style) is most of the work; the tooling is the easy part.
- As the tester, I saw that edge cases (no stock, no volunteers, busy volunteers) define the product as much as the happy path, and that testing the flow turned up configuration bugs early.
- Scoping v1 tightly made the system testable, and an honest limitations list gave a clear roadmap.

Links

Built by Vipin Choudhary with Aditya Mehta, supervised by Ms. Shalini Sharma, JECRC University, Jaipur.
LinkedIn · GitHub